

Lecture 14: Fetch API, JSON, Promises, and Async/Await

Fetching Data with JavaScript:

Data can be fetched from a remote server using the `fetch()` command.

- Executing a standard fetch immediately returns a Promise `<pending>` because the request has not yet completed.
- The `await` keyword is required to pause execution until the data actually arrives.
- Once the data arrives, it is in a stream format and must be parsed into a usable JavaScript object using the `await response.json()` command.
- The `.json()` method also returns a Promise, which is why it requires its own `await` keyword.

```
const response = await fetch("https://api.github.com/users?page=20");
```

```
const data = await response.json();
```

What is JSON?

JSON stands for **JavaScript Object Notation**.

```
const jsonData = {  
  "name": "Rohit",  
  "age": 20,  
  "login": true  
};
```

Why JSON Exists:

- Backend systems are built in various languages (C++, Python, Go, Java), which do not inherently understand JavaScript objects.
- JSON was created as a universally agreed-upon, common format for transferring data between different programming languages.

Note: Despite the name, JSON isn't tied to JavaScript. It's a language-agnostic set of rules for structuring data as a string, and virtually every language has a way to work with it (e.g., C++ can store parsed JSON data inside a map).

Why data is sent as a string?

Data is transmitted in string format rather than as a raw object, since converting a raw object directly would require checking and converting individual characters at a binary level. Sending data as a string first, then parsing it on arrival, is simpler and more portable across languages. Once received, the receiving system determines whether each value should be treated as a number, string, boolean, etc. This is also how tools like ChatGPT typically transmit data.

JSON Rules & Limitations:

- **Supported Types:** JSON supports strings, numbers, booleans, null, objects, and arrays.
- **Unsupported Types:** Unlike standard JavaScript objects, JSON does not support undefined or functions.

Converting Between JSON and Objects:

The data received in a response body is converted into a usable JS object using this underlying approach (which is handled automatically behind the scenes by the `.json()` method).

```
const jsObject = {  
  name: "Rohit",  
  age: 20  
};
```

Object to JSON string:

```
const a = JSON.stringify(jsObject);
```

JSON string to Object:

```
const b = JSON.parse(a);
```

Understanding Promises:

Any response received from a server, whether a success or failure, is wrapped in a Promise.

What is a Promise?

A Promise is an object representing the eventual result of an asynchronous operation. It exists in one of three states:

1. **Pending:** The request has been made, but a result has not arrived yet. Data cannot be read in this state.
2. **Fulfilled:** The request completed successfully, and data is available.
3. **Rejected:** The request failed to reach the server (e.g., no internet, network down, or a malformed URL).

Single-Threaded & Asynchronous Architecture:

- JavaScript is a **single-threaded** language, meaning it can only execute exactly one task at a time.
- To prevent the entire program from freezing while waiting for a remote server to respond, **asynchronous programming** is used to keep the rest of the application running smoothly in the background.
- Any code utilizing the `await` keyword must be wrapped inside a function explicitly marked with the `async` keyword.

```
async function github() {  
  const response = await fetch("https://api.github.com/users?per_page=20");  
  const data = await response.json();  
  console.log(data);  
}  
  
github();
```

Hands-On Projects Using Fetch:

1. Dynamic GitHub Profile Search:

A search feature that dynamically fetches a specific number of GitHub users and renders their avatars and usernames into the UI.

- **Dynamic Fetching:** An event listener reads a numeric input and injects it directly into the fetch() URL.
- **State Reset:** Because a new search count may be smaller than the previous one, the root container (root.textContent = ""); must be cleared out first to prevent old and new results from mixing.
- **Live Search Enhancement:** Attaching the event listener directly to the input field using the 'input' event allows the profiles to update live as the user types.

```
async function github(number = 20) {  
  const response = await fetch(`https://api.github.com/users?per_page=${number}`);  
  const data = await response.json();  
  
  const root = document.getElementById('root');  
  root.textContent = "";  
  
  // DOM Rendering Logic  
}  
  
const input = document.querySelector('input');  
  
input.addEventListener('input', () => {  
  const number = Number(input.value);  
  github(number);  
})
```

2. Weather App:

Fetches and displays the current temperature and weather forecast for a searched city.

- **Logic:** The city name is extracted from the input field and passed as a request parameter to the weather API endpoint.
- **Async Callbacks:** Because await cannot be used directly inside a plain callback, it must be wrapped in an async function (or an async arrow function on the event listener).
- **Empty Inputs:** Prevent invalid API requests by checking for empty strings (if (city === "") return;) before executing the fetch.

```
const search = document.getElementById("weatherButton");  
const input = document.getElementById("cityInput");  
  
search.addEventListener('click', async () => {  
  const city = input.value.trim();  
  
  if (city === "") {  
    return;  
  }  
  
  // Fetch Logic  
})
```